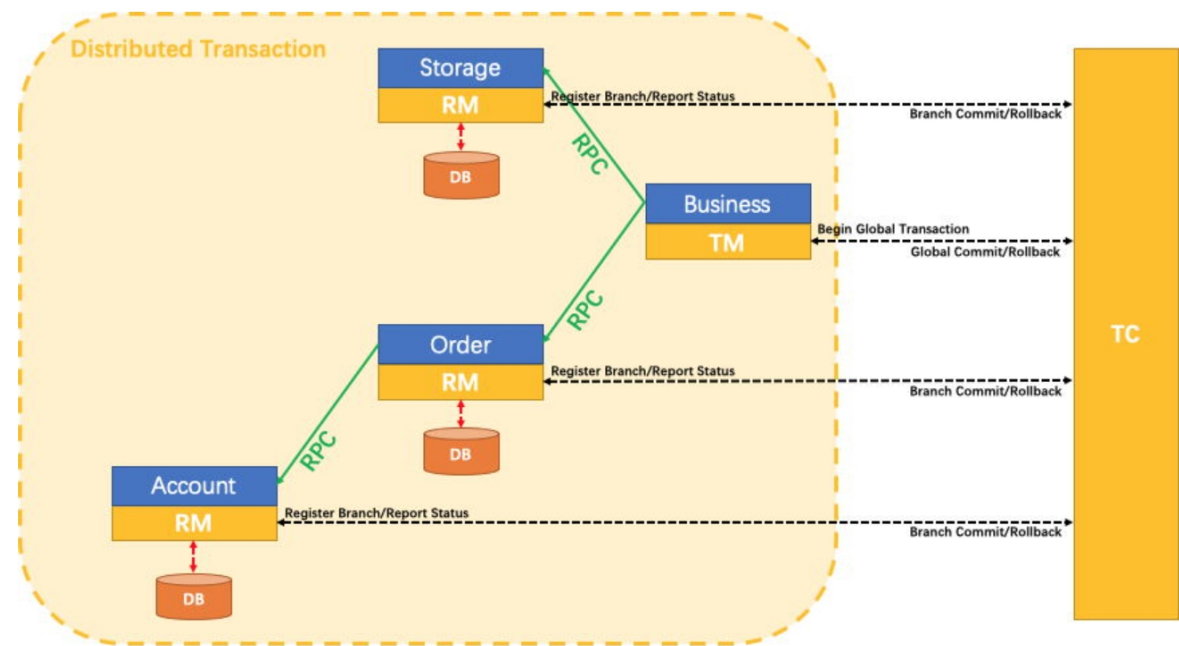


不要在项目中使用分布式事务中间件

什么是分布式事务问题？

做系统肯定要用事务，那你用事务的话，分布式系统之后肯定要用分布式事务

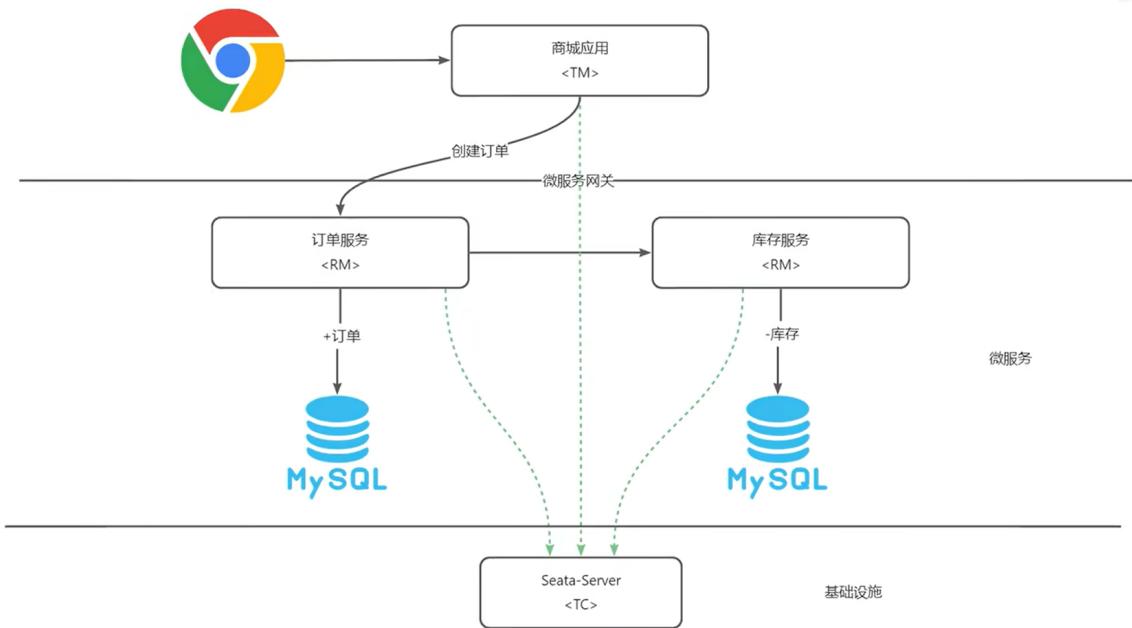


TC (Transaction Coordinator) - 事务协调者，维护全局和分支事务的状态，驱动全局事务提交或回滚。

TM (Transaction Manager) - 事务管理器，定义全局事务的范围：开始全局事务、提交或回滚全局事务。

RM (Resource Manager) - 资源管理器，管理分支事务处理的资源，与TC交谈以注册分支事务和报告分支事务的状态，并驱动分支事务提交或回滚。

引入分布式事务中间件会造成系统大规模耦合的情况



引入分布式事务中间件

优点

将分布式事务进行抽象，独立部署的与业务无关的事务组件进行事务控制

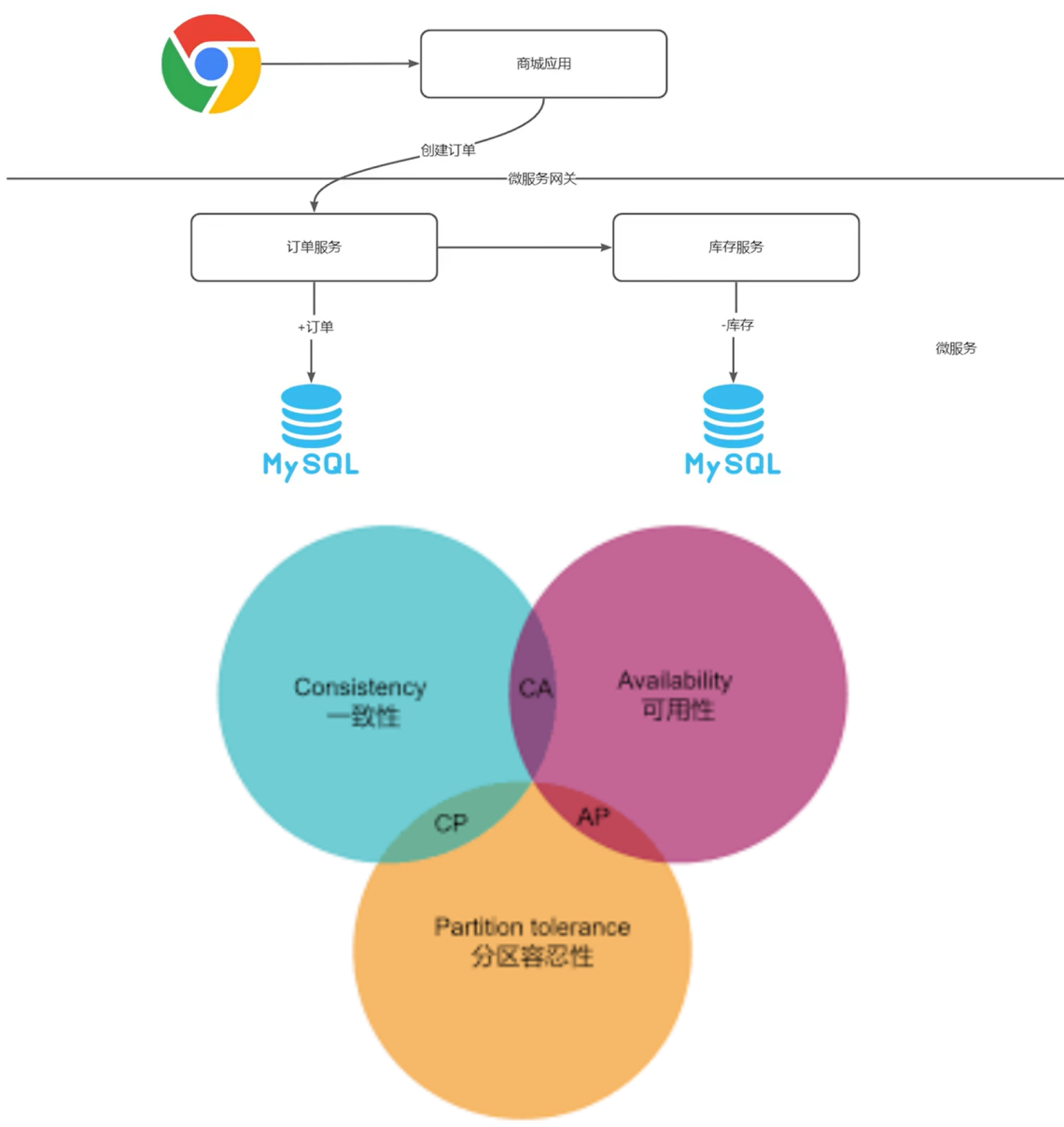
缺点

引入分布式事务中间件会造成系统大规模耦合

TC基础设置对外暴露，既合规，也不安全

用不好(AT、TCC、Saga、XA)就是超低的执行效率，同时也提高了工程师的应用门槛

不用分布式事务中间件，怎么解决？



同步阻塞库：库存扣减不做完不返回结果，库存扣减失败直接返回异常给商城应用。(CP强一致)

异步调用：订单服务异步调用库存服务，订单服务每n秒调用库存服务“出库查询”接口确认订单出库状态。(AP最终一致性)

- 出库成功:订单完成，业务结束
- 因为技术问题出库失败:重试补偿
- 因为业务问题出库失败(比如库存不足):撤销订单，短信通知用户、补偿用户等
- TIPS1：尽量避免分布式事务，初期服务粒度可以粗一些
- TIPS2：对于异步调用，一定要做好资源的校验与锁定，极力避免业务问题产生
- TIPS3：分布式一致性问题会随着调用层级的增加(A->B->C->D->E)难度激增，因此不建议超过3层调用层级，如果出现可以考虑服务聚合缩短调用链(A->(BC)->(DE))

如果一定要用分布式事务中间件呢？

AT 模式

AT 模式是 Seata 创新的一种非侵入式的分布式事务解决方案，Seata 在内部做了对数据库操作的代理层，我们使用 Seata AT 模式时，实际上用的是 Seata 自带的数据源代理 DataSourceProxy，Seata 在这层代理中加入了很多逻辑，比如插入回滚 undo_log 日志，检查全局锁等。

优点：简单易用，不需要改动业务代码，自动完成分布式事务的提交和回滚。

缺点：不适合跨多种存储资源的事务，且在高并发场景下性能可能成问题。

适用场景：主要用于 CRUD（创建、读取、更新、删除）操作较多的业务场景，尤其是当业务逻辑直接操作数据库，并且可以容忍短暂的数据不一致时。AT 模式通过记录数据的前后镜像来实现撤销（回滚）操作，适合于大部分单纯依赖于单个数据库事务的微服务场景。

TCC 模式

TCC 模式是 Seata 支持的一种由业务方细粒度控制的侵入式分布式事务解决方案，是继 AT 模式后第二种支持的事务模式，最早由蚂蚁金服贡献。其分布式事务模型直接作用于服务层，不依赖底层数据库，可以灵活选择业务资源的锁定粒度，减少资源锁持有时间，可扩展性好，可以说是为独立部署的 SOA 服务而设计的。

优点：TCC 完全不依赖底层数据库，能够实现跨数据库、跨应用资源管理，可以提供给业务方更细粒度的控制。

缺点：TCC 是一种侵入式的分布式事务解决方案，需要业务系统自行实现 Try，Confirm，Cancel 三个操作，对业务系统有着非常大的入侵性，设计相对复杂。

适用场景：TCC 模式是高性能分布式事务解决方案，适用于核心系统等对性能有很高要求的场景。

SAGA 模式

适用场景：适用于长事务场景，其中业务流程包含一系列的本地事务，这些本地事务需要按照一定的顺序执行。SAGA 模式通过定义一系列的事务步骤和相对应的补偿操作（回滚操作）来管理事务，适合于微服务架构下的复杂业务流程。

优点：适合长事务处理，可以保证分布式事务的最终一致性。

缺点：需要定义每个步骤的补偿操作，对业务侵入性较高。

XA 模式

XA 模式是从 1.2 版本支持的事务模式。XA 规范是 X/Open 组织定义的分布式事务处理（DTP, Distributed Transaction Processing）标准。Seata XA 模式是利用事务资源（数据库、消息服务等）对 XA 协议的支持，以 XA 协议的机制来管理分支事务的一种事务模式。

优点：可以提供跨多种资源的强一致性事务保证，与 Seata 支持的其它事务模式不同，XA 协议要求事务资源本身提供对规范和协议的支持，所以事务资源（如数据库）可以保障从任意视角对数据的访问有效隔离，满足全局数据一致性。此外的一些优点还包括：

- 业务无侵入：和 AT 一样，XA 模式将是业务无侵入的，不给应用设计和开发带来额外负担。
- 数据库的支持广泛：XA 协议被主流关系型数据库广泛支持，不需要额外的适配即可使用。

缺点：XA prepare 后，分支事务进入阻塞阶段，收到 XA commit 或 XA rollback 前必须阻塞等待。事务资源长时间得不到释放，锁定周期长，而且在应用层上面无法干预，性能差。

适用场景：适用于想要迁移到 Seata 平台基于 XA 协议的老应用，使用 XA 模式将更平滑，还有 AT 模式未适配的数据库应用。且适用于需要强一致性保证的分布式事务，且参与事务的资源管理器（如数据库）支持 XA 协议。XA 模式基于两阶段提交（2PC）协议，适合于跨多种数据库或消息队列等资源的分布式事务。